# DOOM AI - Complete PowerShell Command Cheat Sheet

For the Doom AI Emergency Department Command Center project

Use this document as the day-to-day command reference for launching the UI, running all 16 test cases, checking Gemini/PySide6, compiling modules, and opening generated reports.

## 1. Open the Project Root

```
cd "D:\Accenture Hackathon\Doom_AI_ChatGPT Backup"
```

## 2. Activate the Virtual Environment

```
.\.venv\Scripts\Activate.ps1
```

Expected prompt:

```
(.venv) PS D:\Accenture Hackathon\Doom_AI_ChatGPT Backup>
```

## 3. Fix PowerShell Activation If Blocked

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned
.\.venv\Scripts\Activate.ps1
```

## 4. Verify Which Python Is Running

```
python -c "import sys; print(sys.executable)"
```

The output should point inside .venv, for example:

```
D:\Accenture Hackathon\Doom_AI_ChatGPT Backup\.venv\Scripts\python.exe
```

## 5. Check the Gemini API Key

```
if ($env:GEMINI_API_KEY) {
    "Gemini API key is loaded"
} else {
    "Gemini API key is missing"
}
```

If Windows has the key saved as a User environment variable but the current terminal does not yet see it, load it into the current session:

```
$env:GEMINI_API_KEY = [Environment]::GetEnvironmentVariable("GEMINI_API_KEY", "User")
```

Verify the User variable without printing it into normal logs during routine use:

```
python -c "import os; print('Gemini key detected:', bool(os.getenv('GEMINI_API_KEY')))"
```

## 6. Verify the Gemini SDK and Client

```
python -c "from google import genai; print('Gemini SDK OK')"
```

```
python -c "from google import genai; import os; genai.Client(api_key=os.environ['GEMINI_API_KEY']); print('Gemini clie
```

## 7. Verify PySide6

```
python -c "import PySide6; print('PySide6 OK')"
```

If PySide6 is missing:

```
python -m pip install PySide6
```

## 8. Compile the Main UI Controller

```
python -m py_compile doom/ui/main_window.py
```

No output normally means the syntax check passed.

## 9. Compile the Test-Case UI Service

```
python -m py_compile doom/services/test_case_ui_service.py
```

## 10. Compile the Presentation Result Layer

```
python -m py_compile doom/services/presentation_result.py
```

## 11. Compile the Report Generator

```
python -m py_compile doom/services/result_reporter.py
```

## 12. Compile the Test Runner

```
python -m py_compile test_cases/test_case_runner.py
```

## 13. Launch the Actual DOOM AI UI

```
python -m doom.main
```

Use this command whenever you want to operate the application normally.

## 14. Run the Complete Automated Test-Case Suite

```
python -m test_cases.test_case_runner
```

This runs the full validation suite and updates the reports folder.

## 15. Run an Individual Automated Test Case

```
python -m test_cases.test_05_capacity_transfer
python -m test_cases.test_10_image_pipeline
python -m test_cases.test_11_override_audit
python -m test_cases.test_15_mass_casualty
```

Replace the module name with any of the available test_case files when debugging one case.

## 16. Open the Reports Folder

```
explorer .\reports
```

## 17. Open the Standard Validation Report

```
start .\reports\hackathon_latest.html
```

This is the PASS/FAIL/SKIP validation view.

## 18. Open the Full UI-Equivalent Clinical Report

```
start .\reports\ui_results.html
```

This is the patient-level report that mirrors the canonical result data used by the UI: ESI, criticality, confidence, layer, rationale, routing, transfer, image findings, and related fields.

## 19. Open CSV Reports

```
start .\reports\hackathon_latest.csv
start .\reports\ui_results.csv
```

## 20. Open JSON Reports in VS Code

```
code .\reports\hackathon_latest.json
code .\reports\ui_results.json
```

## 21. Run the Interactive Live UI Test-Case Simulation

There is no separate PowerShell command for the UI simulation. Launch the normal application, then use the Test Case Simulation section inside the UI.

```
python -m doom.main
```

• Select H01-H16 in the Test Case Simulation dropdown.

• Click Load Test Case.

• Confirm the existing Live ED Arrival Stream is populated.

• Click RUN CASE ON LIVE UI.

• The existing batch evaluation pipeline then produces the existing AI Priority Queue / Resource Dispatch results.

## 22. Recommended Normal Startup Sequence

```
cd "D:\Accenture Hackathon\Doom_AI_ChatGPT Backup"
.\.venv\Scripts\Activate.ps1
python -c "import PySide6; print('PySide6 OK')"
python -c "from google import genai; print('Gemini SDK OK')"
python -m doom.main
```

## 23. Recommended Pre-Hackathon Validation Sequence

```
cd "D:\Accenture Hackathon\Doom_AI_ChatGPT Backup"
.\.venv\Scripts\Activate.ps1
python -m py_compile doom/ui/main_window.py
python -m py_compile doom/services/test_case_ui_service.py
python -m py_compile doom/services/presentation_result.py
python -m py_compile doom/services/result_reporter.py
python -m py_compile test_cases/test_case_runner.py
python -m test_cases.test_case_runner
start .\reports\ui_results.html
python -m doom.main
```

## 24. Judge Demonstration Sequence

Recommended live demo flow:

• Launch Doom AI with python -m doom.main.

• Select H15 - Mass Casualty Surge.

• Load the case and show the 10 arrivals in the existing Live ED Arrival Stream.

• Run the case on the live UI and show the ESI 1-5 classification, criticality, confidence, active layer, resource dispatch, and transfer information.

• Optionally demonstrate H05 - Full Capacity + Transfer.

• Demonstrate H10 - Clinical Image Ingestion & Findings.

• Demonstrate H07 - Demographic Calibration.

• Demonstrate H08 - Pessimistic Safety Floor.

• After the demo, run the automated test suite and open ui_results.html for the patient-level evidence report.

## 25. Quick Command Table

| Purpose | Command |
|---|---|
| Activate environment | `.\.venv\Scripts\Activate.ps1` |
| Launch UI | `python -m doom.main` |
| Run all 16 test cases | `python -m test_cases.test_case_runner` |
| Compile UI controller | `python -m py_compile doom/ui/main_window.py` |
| Open reports folder | `explorer .\reports` |

| Purpose | Command |
| --- | --- |
| Open validation report | `start .\reports\hackathon_latest.html` |
| Open full clinical report | `start .\reports\ui_results.html` |
| Check Gemini key | `python -c "import os; print(bool(os.getenv(\'GEMINI_API_KEY\')))"` |
| Check PySide6 | `python -c "import PySide6; print(\'PySide6 OK\')"` |

## Important operational note

The UI simulation and automated reports use the Doom AI application pipeline. The test-case layer is for validation/demo purposes and does not replace clinician review or clinical validation.